

Module Title:	<i>H.Dip. in Computing</i>
Assessment Title:	Algorithms and Constructs
Lecturer Name:	Taufique Ahmed
Student Full Name:	Jilsa john
Student Number:	2025726
Assessment Due Date:	23/05/2026
Date of Submission:	23/05/2026

Declaration

By submitting this assessment, I confirm that I have read the CCT policy on Academic Misconduct and understand the implications of submitting work that is not my own or does not appropriately reference material taken from a third party or other source. I declare it to be my own work and that all material from third parties has been appropriately referenced. I further confirm that this work has not previously been submitted for assessment by myself or someone else in CCT College Dublin or any other higher education institution.

1. INTRODUCTION

This report presents an Organization Management System designed for a Department Store. The system manages employee records, implements hierarchical structures, and provides efficient searching and sorting capabilities.

2. ALGORITHM JUSTIFICATION

After analyzing various sorting algorithms, I selected **Merge Sort** for these key reasons:

The sorting algorithm chosen for the Department Store Organisation System is Recursive Merge Sort, implemented in the `SortAlgorithms.mergeSort()` static method. The decision was based on several practical reasons specific to this system's requirements, compared against the two main alternatives: Quick Sort and Bubble Sort.

The most important reason for choosing Merge Sort is its guaranteed $O(n \log n)$ time complexity in all cases — best, average, and worst. Quick Sort, which is often considered a fast alternative, degrades to $O(n^2)$ in its worst case when the data is already sorted or nearly sorted. Since the employee list in this system may already be partially ordered after previous sorts, Quick Sort's worst-case scenario is a real risk. Merge Sort eliminates this concern entirely by maintaining consistent performance regardless of the input arrangement.

Secondly, Merge Sort is a stable sorting algorithm, meaning it preserves the relative order of equal elements. This is important when sorting employee records by name, as employees sharing the same surname will maintain their original relative order. Quick Sort is not stable by default, which could result in inconsistent ordering of duplicate entries.

Bubble Sort was also considered but immediately eliminated. Its $O(n^2)$ complexity makes it entirely unsuitable for a dataset of 46+ employees that could grow significantly. It would produce noticeably slower performance even at moderate scale and does not meet the non-functional requirement NFR-1, which requires sorting 1000 records in under 2 seconds.

Finally, Merge Sort is naturally recursive by design, making it a perfect fit for the assignment's recursive algorithm requirement. Unlike iterative approaches, the recursive divide-and-conquer structure of Merge Sort is clean, readable, and easy to verify — aligning well with the code quality standards required for this submission (Cormen et al., 2022; Knuth, 2011).

Algorithm	Time Complexity (Worst)	Stable?	Recursive?
Merge Sort Chosen	$O(n \log n)$ — always	Yes	Yes
Quick Sort	$O(n^2)$ worst case	NO	Yes
Bubble Sort	$O(n^2)$ always	Yes	No

REFERENCES

1. Cormen, T. H., et al. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
2. Oracle Java Documentation: <https://docs.oracle.com/javase/>

